

Dima Beauty Center

1. Introduction

From the very beginning, I have loved being a programmer because it gives me the power to turn any idea that comes to my mind into reality. This project — **Dima Beauty Center** — is a reflection of that passion. It is not just a technical exercise, but also a vision for my future beauty center. By combining creativity with programming, I designed a complete management system that could one day serve as the digital backbone of my business.

2. Goal of the Project

The goal of this project is to create a full-stack web application that allows administrators to manage all aspects of a beauty center, including:

- Authentication: Secure login/logout for admin users.
- Appointments: Scheduling and managing client bookings.
- Clients: Adding, updating, and deleting client records.
- Employees: Managing staff details and specialties.
- Services: Maintaining a catalog of beauty services with prices.

This system ensures smooth operations, prevents scheduling conflicts, and provides a centralized dashboard for managing the business.

4. Project Structure

Layer	Folder	Purpose
Main App	DimaBeautyApplication.java	Entry point for Spring Boot
Controllers	com.example.dimaBeauty.	REST endpoints for appointments, clients, employees, services, authentication
Entities	com.example.dimaBeauty.	Database models (Appointment, Client, Employee, Service, User)

Repositories	com.example.dimaBeauty.	Interfaces for CRUD operations
Frontend	scr/main/resources/static.	HTML, CSS, JS files for UI
Config	application.pproperties	Database and server configuration

Entity Relationships

The system is centered around the Appointment entity, which connects clients, employees, and services. Each appointment belongs to one client, one employee, and one service. However, each client, employee, and service can be linked to multiple appointments. This creates a many-to-one relationship from appointments to each of these entities. The User entity is separate and is only used for authentication, ensuring that only authorized admins can access the system

User

- **Fields:** id, username, password
- **Purpose:** Represents admin users who can log in to the system.
- **Relationships:**
- Independent entity (not directly tied to appointments, clients, employees, or services).
- Used only for authentication.

Client

- **Fields:** id, name, phone
- **Relationships:**
- **One-to-Many with Appointment**
- A client can have **multiple appointments**.
- Defined in Client.java with:

```
@OneToMany(mappedBy = "client", cascade = CascadeType.ALL) private List appointments;
```

Employee

- **Fields:** id, name, specialty
- **Relationships:**
- **One-to-Many with Appointment**
- An employee can handle **multiple appointments**.
- Defined in Employee.java with:

```
@OneToMany(mappedBy = "employee", cascade = CascadeType.ALL) private List appointments;
```

Serv (Service)

- **Fields:** id, name, price
- **Relationships:**
- **One-to-Many with Appointment**
- A service can be booked in **multiple appointments**.
- Defined in Serv.java with:

```
@OneToMany(mappedBy = "serv", cascade = CascadeType.ALL) private List appointments;
```

Appointment

- **Fields:** id, date, time, client, employee, serv
- **Relationships:**
- **Many-to-One with Client**
 - Each appointment belongs to **one client**.
- **Many-to-One with Employee**
 - Each appointment is handled by **one employee**.
- **Many-to-One with Serv (Service)**
 - Each appointment is for **one service**.
- Defined in Appointment.java with:

```
@ManyToOne private Client client; @ManyToOne private Employee employee;  
@ManyToOne private Serv serv;
```

- Client–Appointment: One client → many appointments.
- Employee–Appointment: One employee → many appointments.
- Service–Appointment: One service → many appointments.
- User: Independent, used for login only.

Frontend–Backend Connection

The frontend connects to the backend using REST APIs. Each HTML page has a JavaScript file that sends HTTP requests (GET, POST, DELETE) to the backend controllers. The backend receives these requests, processes them using repositories and entities, and returns JSON responses. The frontend then updates the page dynamically with this data. For example, when creating an appointment, the frontend sends a POST request with client, service, employee, date, and time. The backend saves it in the database and responds, and the frontend refreshes the table to show the new appointment

Module Documentation

Authentication

- **Login Page (login.html)** → Sends credentials to /auth/login.

- **Backend (AuthController)** → Validates username/password via UserRepository.
- **Frontend (login.js)** → Stores login state in localStorage and redirects to dashboard.
- **Logout** → Clears login state and redirects to login page.

Appointments

- **Frontend (appointments.html + appointments.js)** → Create, view, delete appointments.
- **Backend (AppointmentController)** → Handles CRUD, prevents double-booking.
- **Entity (Appointment)** → Links client, employee, service, date, time.

Clients

- **Frontend (clients.html + clients.js)** → Add, update, delete clients.
- **Backend (ClientController)** → CRUD operations via ClientRepository.
- **Entity (Client)** → Stores client name, phone, appointments.

Employees

- **Frontend (employees.html + employees.js)** → Add, update, delete employees.
- **Backend (EmployeeController)** → CRUD operations via EmployeeRepository.
- **Entity (Employee)** → Stores employee name, specialty, appointments.

Services

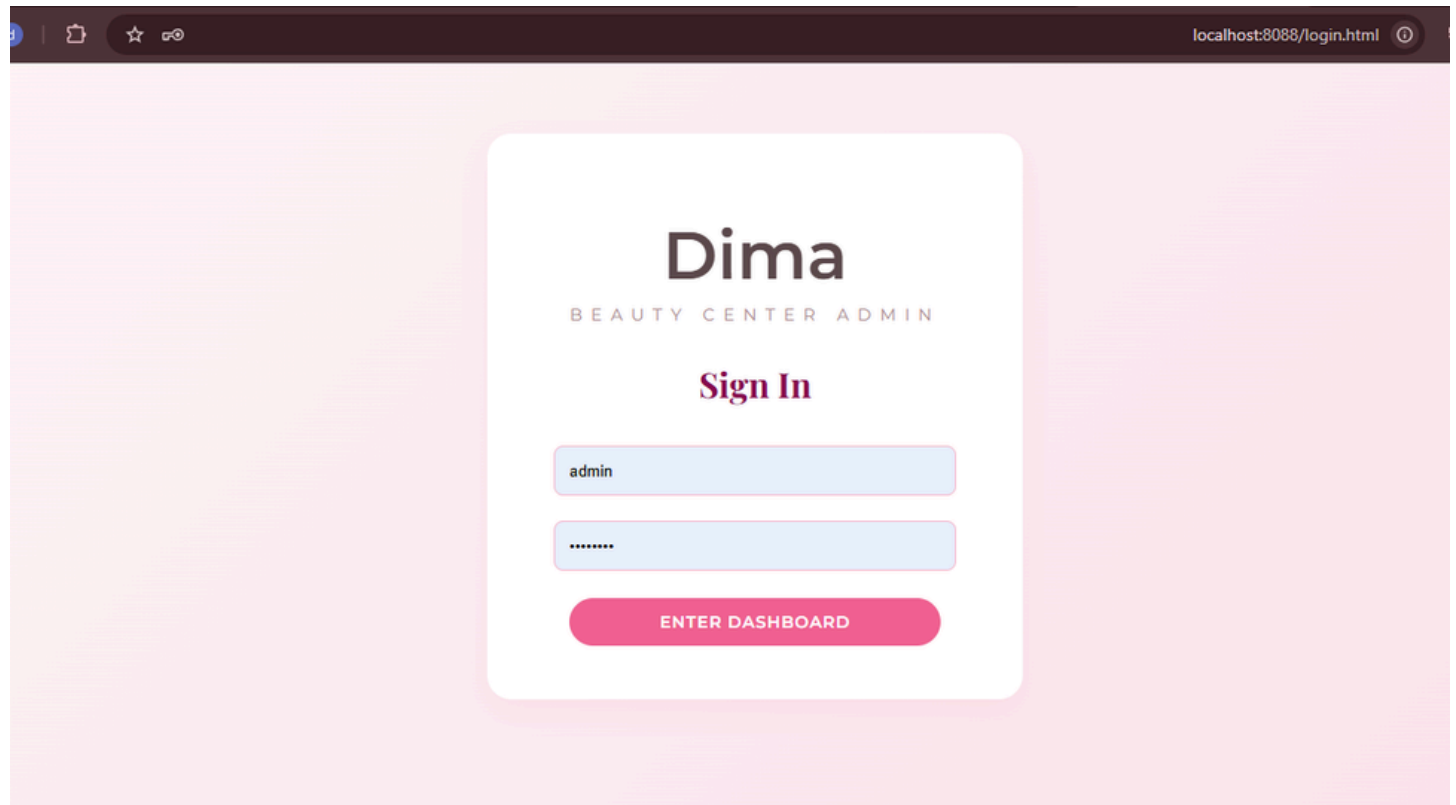
- **Frontend (services.html + services.js)** → Add, update, delete services.
- **Backend (ServController)** → CRUD operations via ServRepository.
- **Entity (Serv)** → Stores service name, price, appointments.

Page Explanations

- **Login Page:** Authenticates admin users.
- **Dashboard (Index Page):** Central hub with navigation and gallery.
- **Appointments Page:** Manage bookings between clients and employees.
- **Clients Page:** Manage client records.
- **Employees Page:** Manage staff details and specialties.
- **Services Page:** Manage catalog of services and prices.

ScreenShots From the code

login.html



```
employees.html index.html login.html X
1 |<!DOCTYPE html>
2 |<html lang="en">
3 |<head>
4 |  <meta charset="UTF-8">
5 |  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 |  <title>Dima Beauty | Admin Login</title>
7 |  <link rel="stylesheet" href="css/style.css">
8 |</head>
9 |<body class="login-page">
10 |
11 |<div class="login-wrapper">
12 |  <div class="form-box">
13 |    <div class="login-header">
14 |      <h1>Dima</h1>
15 |      <p class="subtitle">Beauty Center Admin</p>
16 |    </div>
17 |    <h2>Sign In</h2>
18 |
19 |    <div class="input-group">
20 |      <input id="username" placeholder="Username">
21 |      <input id="password" type="password" placeholder="Password">
22 |    </div>
23 |
24 |    <button class="login-btn" onclick="login()">Enter Dashboard</button>
25 |
26 |    <p id="msg" class="error-msg"></p>
27 |  </div>
28 |</div>
29 |
30 |<script src="js/login.js"></script>
31 |</body>
32 |</html>
```

```
employees.js login.js X
1 |
2 |const username = document.getElementById("username");
3 |const password = document.getElementById("password");
4 |const msg = document.getElementById("msg");
5 |
6 |//LOGIN FUNCTION
7 |function login() {
8 |  fetch("http://localhost:8088/auth/login", {
9 |    method: "POST",
10 |    headers: { "Content-Type": "application/json" },
11 |    body: JSON.stringify({
12 |      username: username.value,
13 |      password: password.value
14 |    })
15 |  })
16 |  .then(res => res.text())
17 |  .then(result => {
18 |    if (result === "LOGIN_SUCCESS") {
19 |      localStorage.setItem("loggedIn", "true");
20 |      window.location.href = "index.html";
21 |    } else {
22 |      msg.innerText = "Invalid username or password";
23 |    }
24 |  });
25 |}
26 |
27 |function logout() {
28 |  localStorage.removeItem("loggedIn");
29 |  window.location.href = "login.html";
30 |}
31 |
32 |
```

```
User.java X
1 |package com.example.dimaBeauty.Entity;
2 |
3 |
4 |import jakarta.persistence.*;
5 |
6 |@Entity
7 |public class User {
8 |
9 |  @Id
10 |  @GeneratedValue(strategy = GenerationType.IDENTITY)
11 |  private Long id;
12 |
13 |  private String username;
14 |  private String password;
15 |
16 |  // getters & setters
17 |  public Long getId() { return id; }
18 |  public void setId(Long id) { this.id = id; }
19 |
20 |  public String getUsername() { return username; }
21 |  public void setUsername(String username) { this.username = username; }
22 |
23 |  public String getPassword() { return password; }
24 |  public void setPassword(String password) { this.password = password; }
25 |}
```

```
DimaBeautyApplication.java index.html login.html login.js AuthController.java X
1 |package com.example.dimaBeauty.Controller;
2 |
3 |
4 |import org.springframework.beans.factory.annotation.Autowired;
5 |
6 |@RestController
7 |@RequestMapping("/auth")
8 |@CrossOrigin(origins = "**")
9 |public class AuthController {
10 |
11 |  @Autowired
12 |  UserRepository userRepo;
13 |
14 |  @PostMapping("/login")
15 |  public String login(@RequestBody User u) {
16 |
17 |    User dbUser = userRepo.findByUsername(u.getUsername());
18 |
19 |    if (dbUser == null)
20 |      return "USER_NOT_FOUND";
21 |
22 |    if (!dbUser.getPassword().equals(u.getPassword()))
23 |      return "WRONG_PASSWORD";
24 |
25 |    return "LOGIN_SUCCESS";
26 |  }
27 |
28 |}
29 |
30 |
31 |
32 |
```

```

DimaBeautyApplication.java  index.html  login.html  login.js  User.java
1 package com.example.dimaBeauty.Entity;
2
3
4 import jakarta.persistence.*;
5
6 @Entity
7 public class User {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    private String username;
14    private String password;
15
16    // getters & setters
17    public Long getId() { return id; }
18    public void setId(Long id) { this.id = id; }
19
20    public String getUsername() { return username; }
21    public void setUsername(String username) { this.username = username; }
22
23    public String getPassword() { return password; }
24    public void setPassword(String password) { this.password = password; }
25 }

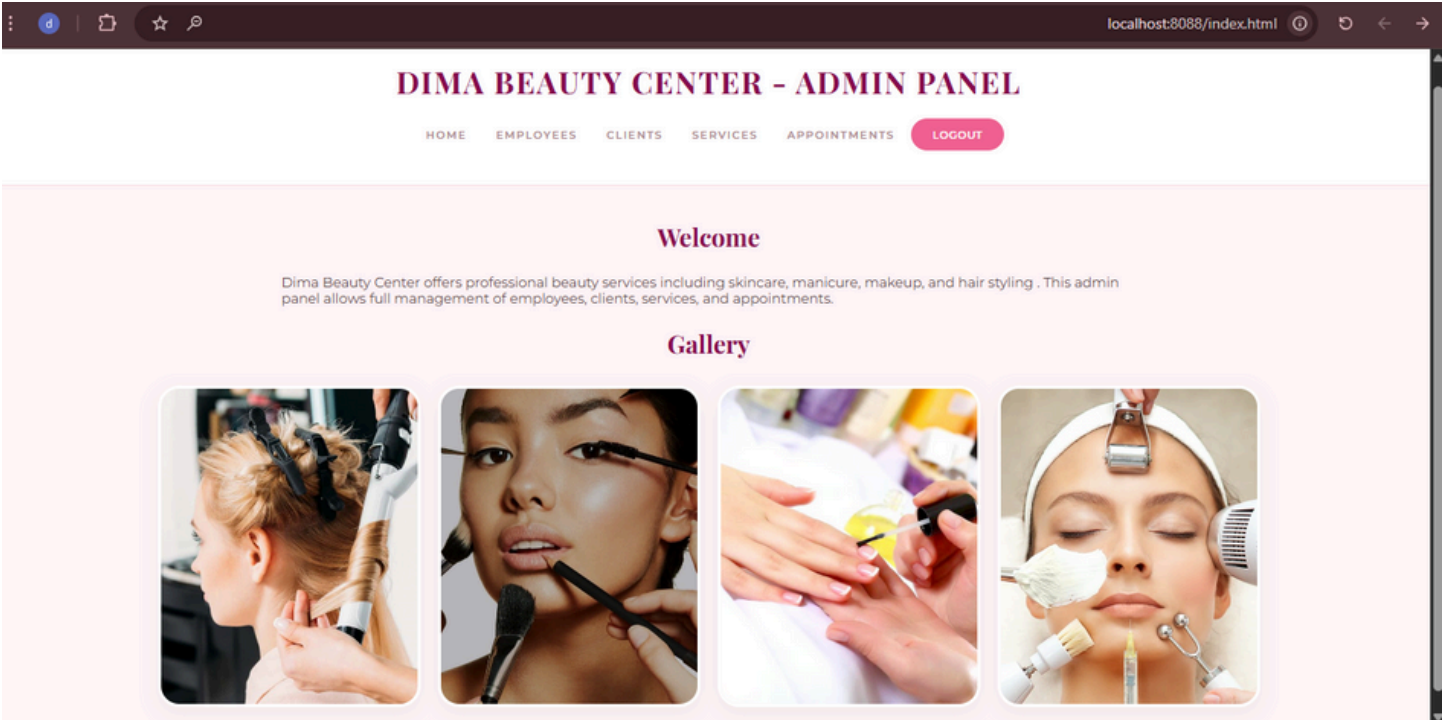
```

```

ServRepository.java  UserRepository.java
1 package com.example.dimaBeauty.Repository;
2
3
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6
7 public interface UserRepository extends JpaRepository<User, Long> {
8     User findByusername(String username);
9 }

```

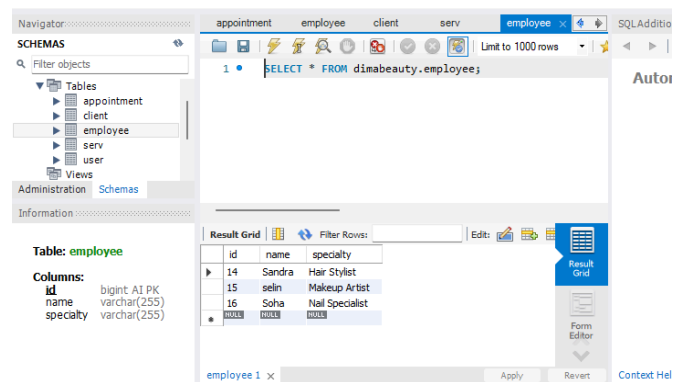
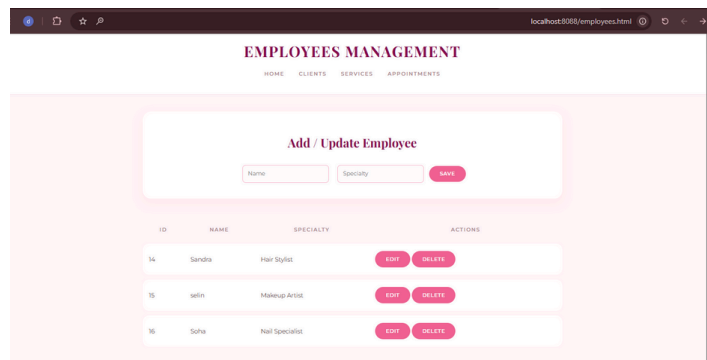
the index page



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Dima Beauty Center</title>
5   <link rel="stylesheet" href="css/style.css">
6 </head>
7 <body>
8
9   <header>
10    <h1>DIMA BEAUTY CENTER - ADMIN PANEL</h1>
11    <nav>
12      <a href="index.html">Home</a>
13      <a href="employees.html">Employees</a>
14      <a href="clients.html">Clients</a>
15      <a href="services.html">Services</a>
16      <a href="appointments.html">Appointments</a>
17
18      <button class="logout-btn" onclick="logout()">Logout</button>
19    </nav>
20  </header>
21
22  <div class="container">
23    <h2>Welcome</h2>
24
25    <p>
26      Dima Beauty Center offers professional beauty services including skincare,
27      manicure, makeup, and hair styling . This admin panel allows full management of
28      employees, clients, services, and appointments.
29    </p>
30
31    <h2>Gallery</h2>
32    <div class="image-grid">
```

```
19    </nav>
20  </header>
21
22  <div class="container">
23    <h2>Welcome</h2>
24
25    <p>
26      Dima Beauty Center offers professional beauty services including skincare,
27      manicure, makeup, and hair styling . This admin panel allows full management of
28      employees, clients, services, and appointments.
29    </p>
30
31    <h2>Gallery</h2>
32    <div class="image-grid">
33
34      
35      
36      
37      
38    </div>
39  </div>
40
41  <script>
42
43    if (localStorage.getItem("loggedIn") != "true") {
44      window.location.href = "login.html";
45    }
46  </script>
47
48  <script src="js/login.js"></script>
49 </body>
50 </html>
```

the employees management page



1. Purpose

The Employees Management module allows administrators to:

- Add new employees with their name and specialty.
- Update existing employee details.
- View all employees in a table.
- Delete employees when necessary.

This ensures the beauty center maintains an accurate record of staff and their specialties.

2. Frontend (HTML + JS)

HTML (employees.html)

- Form Section
 - Hidden input for id (used when editing).
 - Text inputs for Name and Specialty.
 - "Save" button triggers saveEmployee().
- Table Section
 - Displays all employees with columns: ID, Name, Specialty, Actions.
 - Each row includes "Edit" and "Delete" buttons.
- Navigation
 - Links to other modules: Home, Clients, Services, Appointments.

JavaScript (employees.js)

- API Base URL:
 - `const API = "http://localhost:8088/employees";`
- Load Employees (loadEmployees)
 - Sends GET /employees request.
 - Renders all employees into the table.
- Save Employee (saveEmployee)
 - If id is empty → sends POST /employees (create new).
 - If id has a value → sends PUT /employees/{id} (update existing).
 - After saving, clears form and reloads table.
- Edit Employee (edit)
 - Populates form fields with selected employee's data.
 - Allows updating via PUT.

- Delete Employee (del)
- Sends DELETE /employees/{id} request.
- Reloads table after deletion.

3. Backend (Spring Boot)

Entity (Employee.java)

- Represents an employee record in the database.
- Fields:
 - id (Primary Key)
 - name (String)
 - specialty (String)
- Relationships:
 - One-to-Many with Appointment (an employee can have multiple appointments).

Controller (EmployeeController.java)

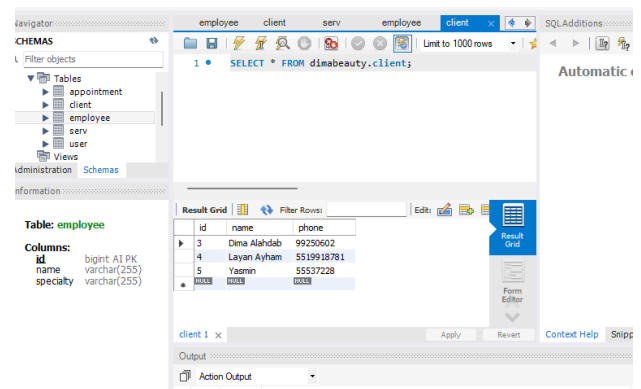
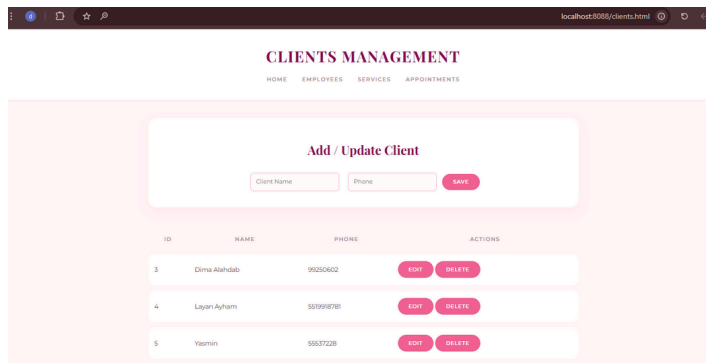
- Base Path: /employees
- Endpoints:
 - GET /employees → Returns all employees.
 - POST /employees → Creates a new employee.
 - PUT /employees/{id} → Updates existing employee.
 - DELETE /employees/{id} → Deletes employee by ID.

Repository (EmployeeRepository.java)

- Extends JpaRepository.
- Provides CRUD operations automatically.

4. Frontend-Backend Connection Flow

1. User Action: Admin enters employee name and specialty → clicks "Save".
2. Frontend Request:
 - JS sends either:
 - POST /employees with JSON body {name, specialty} (new employee).
 - PUT /employees/{id} with JSON body {name, specialty} (update employee).
3. Backend Processing:
 - EmployeeController receives request → saves/updates employee in DB via EmployeeRepository.
4. Response:
 - Backend returns the saved employee object (including ID).
5. Frontend Update:
 - JS reloads table by calling GET /employees again.
6. Deleting Employee:
 - Clicking "Delete" sends DELETE /employees/{id} → backend removes record → frontend reloads table.

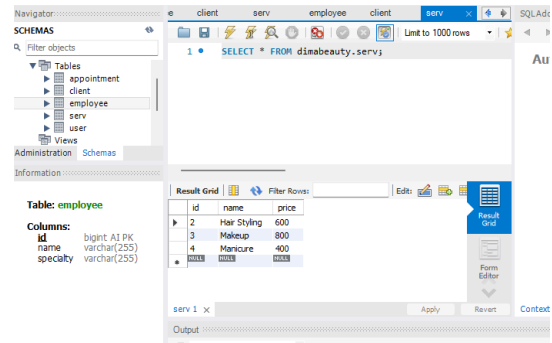
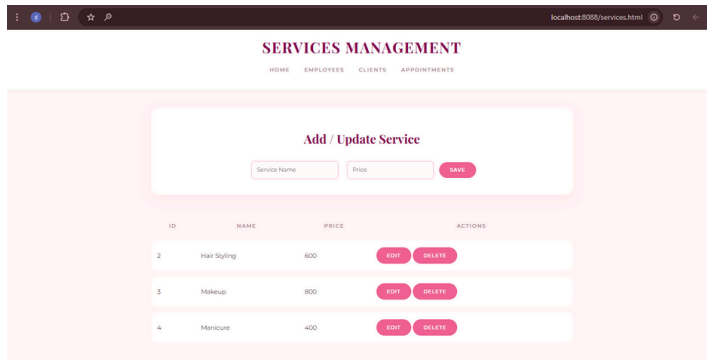


Services Page

- **Purpose:** Manage the catalog of services offered by the beauty center.
- **Features:**
 - Add new services with name and price.
 - Edit existing service details.
 - Delete services.
 - View all services in a table.
- **Backend Connection:** Uses REST API endpoints (/services) to fetch, create, update, and delete records.
- **Data Flow:** Frontend JS sends requests → backend controller processes → repository interacts with database → JSON response updates UI.

Frontend-Backend Connection Flow

1. User Action: Admin enters service name and price → clicks "Save".
2. Frontend Request:
 - JS sends either:
 - POST /services with JSON body {name, price} (new service).
 - PUT /services/{id} with JSON body {name, price} (update service).
3. Backend Processing:
 - ServController receives request → saves/updates service in DB via ServRepository.
4. Response:
 - Backend returns the saved service object (including ID).
5. Frontend Update:
 - JS reloads table by calling GET /services again.
6. Deleting Service:
 - Clicking "Delete" sends DELETE /services/{id} → backend removes record → frontend reloads table.



1. Purpose

The **Services Management** module allows administrators to:

- Add new services offered by the beauty center (e.g., hair styling, makeup, skincare).
- Update service details such as name and price.
- View all services in a table.
- Delete services when necessary.

This ensures the beauty center maintains an accurate catalog of services for clients to book.

2. Frontend (HTML + JS)

HTML (services.html)

- **Form Section**
 - Hidden input for id (used when editing).
 - Text inputs for **Service Name** and **Price**.
 - "Save" button triggers saveService().
- **Table Section**
 - Displays all services with columns: ID, Name, Price, Actions.
 - Each row includes "Edit" and "Delete" buttons.
- **Navigation**
 - Links to other modules: Home, Employees, Clients, Appointments.

JavaScript (services.js)

- **API Base URL:**
- const API = "http://localhost:8088/services";
- **Load Services (loadServices)**
 - Sends GET /services request.
 - Renders all services into the table.
- **Save Service (saveService)**
 - If id is empty → sends POST /services (create new).
 - If id has a value → sends PUT /services/{id} (update existing).
 - After saving, clears form and reloads table.
- **Edit Service (edit)**
 - Populates form fields with selected service's data.
 - Allows updating via PUT.

- **Delete Service (del)**
- Sends DELETE /services/{id} request.
- Reloads table after deletion.

3. Backend (Spring Boot)

Entity (Serv.java)

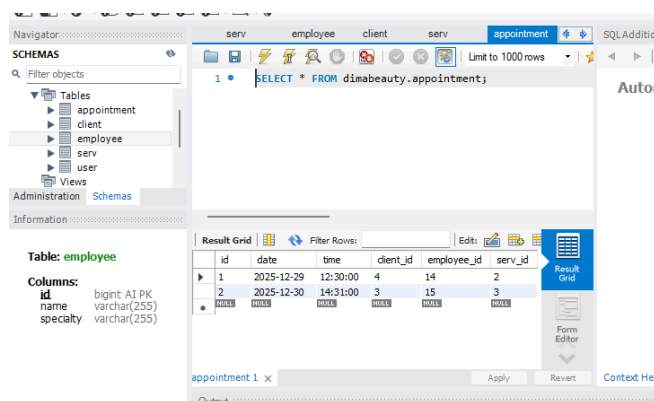
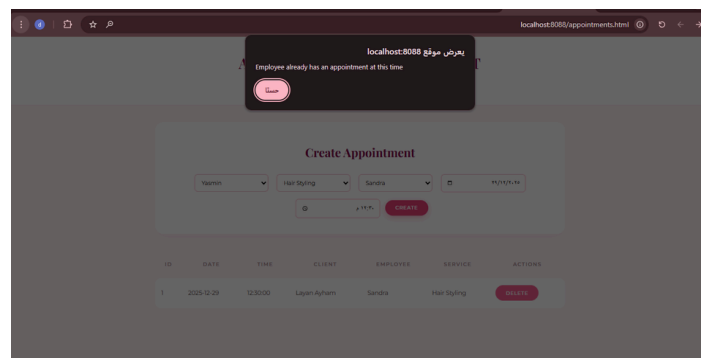
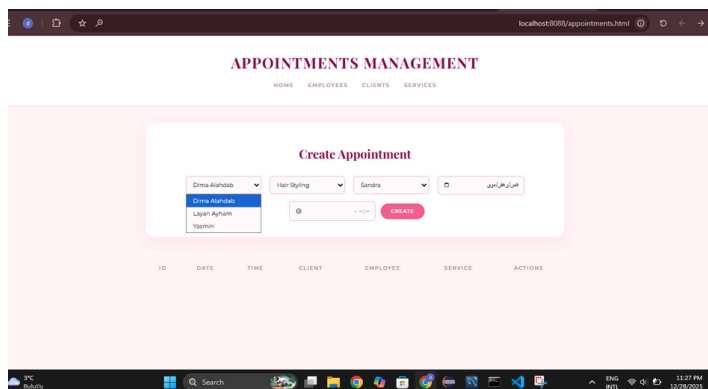
- Represents a service record in the database.
- Fields:
 - id (Primary Key)
 - name (String)
 - price (double)
- Relationships:
- One-to-Many with Appointment (a service can be booked in multiple appointments).

Controller (ServController.java)

- **Base Path:** /services
- **Endpoints:**
- GET /services → Returns all services.
- POST /services → Creates a new service.
- PUT /services/{id} → Updates existing service.
- DELETE /services/{id} → Deletes service by ID.

Repository (ServRepository.java)

- Extends JpaRepository.
- Provides CRUD operations automatically.



1. Purpose

The **Appointments Management** module allows administrators to:

- Create new appointments by selecting a client, service, employee, date, and time.
- View all scheduled appointments in a table.
- Delete existing appointments.

This ensures the beauty center can efficiently manage bookings and avoid scheduling conflicts.

2. Frontend (HTML + JS)

HTML (appointments.html)

- **Form Section**
 - Dropdowns for selecting **Client**, **Service**, and **Employee**.
 - Date and Time inputs.
 - "Create" button triggers addAppointment().
- **Table Section**
 - Displays all appointments with columns: ID, Date, Time, Client, Employee, Service, Actions.
 - Each row includes a "Delete" button.
- **Navigation**
- Links to other modules: Home, Employees, Clients, Services.

JavaScript (appointments.js)

- **API Base URL:**
- const API = "http://localhost:8088/appointments";

- **Dropdown Loading (loadSelect)**

- Fetches data from /clients, /services, /employees.
- Populates elements dynamically.
- **Load Appointments (loadAppointments)**
 - Fetches all appointments from /appointments.
 - Renders them into the table.
- **Add Appointment (addAppointment)**
 - Sends a POST request to /appointments/{clientId}/{servId}/{employeeId}.
 - Includes JSON body with date and time.
 - Reloads the table after success.
- **Delete Appointment (del)**
 - Sends a DELETE request to /appointments/{id}.
 - Reloads the table after deletion.

3. Backend (Spring Boot)

- **Entity (Appointment.java)**
 - Represents an appointment record in the database.
 - Fields:
 - id (Primary Key)
 - client (Many-to-One relationship)
 - serv (Many-to-One relationship)
 - employee (Many-to-One relationship)
 - date (LocalDate)
 - time (LocalTime)
- This ensures each appointment is linked to a specific client, service, and employee.
- **Controller (AppointmentController.java)**
 - **Base Path:** /appointments
 - **Endpoints:**
 - POST /appointments/{clientId}/{servId}/{employeeId}
 - Creates a new appointment.
 - Checks for conflicts (same employee, same date/time).
 - Returns error if conflict exists.
 - GET /appointments
 - Returns all appointments.
 - DELETE /appointments/{id}
 - Deletes an appointment by ID.
- **Conflict Handling**
- Before saving, the controller checks:
- `boolean conflict = !appRepo.findByEmployeeIdAndDateAndTime(employeeId, a.getDate(), a.getTime()).isEmpty();`
- `!!`
- If conflict exists → returns 400 Bad Request with message:
"Employee already has an appointment at this time"

4. Frontend-Backend Connection Flow

1. **User Action:** Admin selects client, service, employee, date, time → clicks "Create".
2. **Frontend Request:**
JS sends POST request with JSON body {date, time} to backend endpoint /appointments/{clientId}/{servId}/{employeeId}.

3. **Backend Processing:**

Controller validates → checks conflict → saves appointment in DB.

4. **Response:**

Backend returns success message → frontend reloads table with updated data.

5. **Viewing Appointments:**

JS calls GET /appointments → renders table.

6. **Deleting Appointment:**

Clicking "Delete" sends DELETE /appointments/{id} → backend removes record → frontend reloads table.

Conclusion

This project demonstrates how programming can transform ideas into reality. By building **Dima Beauty Center**, I combined my passion for technology with my vision of owning a beauty center in the future. The system is scalable, modular, and practical — showing how a real business could be managed digitally.

It is not just code; it is the foundation of a dream.